

Hacking Articles

Raj Chandel's Blog

Menu

[Home](#) » [Nmap](#) » [Nmap for Pentester: Host Discovery](#)

Nmap

Nmap for Pentester: Host Discovery

December 16, 2020 By Raj

Overview

This article examines Nmap host discovery end-to-end and shows how a packet-filtering firewall answers it at every step. It walks through each discovery probe Nmap offers — the default combination, the TCP SYN and ACK pings, the ICMP echo and timestamp pings, the UDP ping, and the IP-protocol ping — and demonstrates, with live terminal output and packet captures, exactly how each one confirms that a host is alive. For every technique, the target then deploys a focused iptables rule that neutralises it, and the scan is repeated to prove the rule works. The article closes by reaching the attacker's final fallback, the `-Pn` option, and then consolidates the defensive findings into a practical set of mitigation strategies.

Table of Contents:

- Introduction
- Understanding TCP Flags
- Host Discovery Techniques
- Lab Environment
- Establishing a Baseline: Default Host Discovery
- Blocking the Default Probes
- TCP SYN Ping (`-PS`)
- TCP ACK Ping (`-PA`)
- ICMP Echo Ping (`-PE`)
- ICMP Timestamp Ping (`-PP`)
- UDP Ping (`-PU`)
- IP-Protocol Ping (`-PO`)

- ARP Ping (-PR)
- SCTP INIT Ping (-PY)
- When Every Probe Fails: Skip Discovery (-Pn)
- Mitigation Strategies
- Conclusion

Introduction

Network reconnaissance begins with host discovery: before an attacker enumerates services, they must first confirm that a target is alive. Nmap performs this step automatically, yet a defender can shape its outcome. This article demonstrates how Nmap establishes liveness through several distinct probe types and how an **iptables** firewall on the target neutralises each technique in turn.

Because the attacker and target share a layer-2 segment, Nmap would normally resolve liveness with an ARP request, which no IP-layer rule can suppress. To expose the higher-layer probes to the firewall — and make the experiment meaningful — every scan carries the `-disable-arp-ping` flag, while the legacy `-sP` syntax requests host discovery only. As each probe is blocked, Nmap's own output repeatedly suggests `-Pn`; the article follows that thread to its logical end.

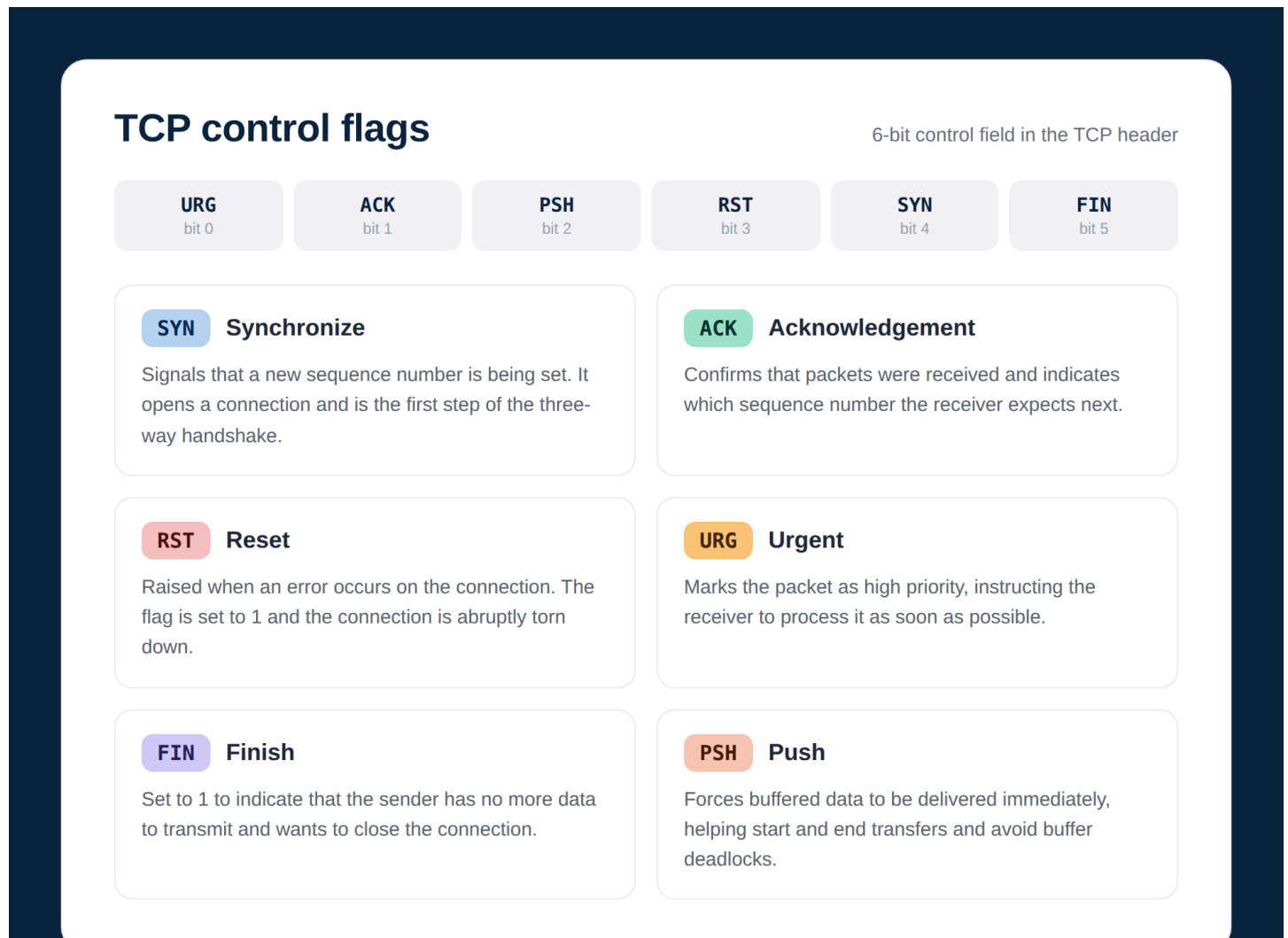
Understanding TCP Flags

TCP control flags are single-bit fields in the TCP header that signal the state or intent of a packet — essentially on/off switches (set to 1 or 0) that tell the receiving end how to interpret and handle the segment. They coordinate everything about a TCP connection: opening it, acknowledging data, prioritising delivery, and closing or aborting it. Because TCP is a connection-oriented, reliable protocol, both ends need a way to communicate control information alongside the actual data. The flags are that mechanism. When a flag bit is set to 1, it's "raised" or active; when it's 0, it's inactive. Multiple flags can be set at once in a single packet — for example, a SYN-ACK packet during the handshake has both the SYN and ACK flags set simultaneously. Let us briefly understand each flag, which is as follows:

- **SYN (Synchronise)** — Signals that a new sequence number is being established. It opens a connection and is the first step of the three-way handshake (the SYN → SYN-ACK → ACK sequence that starts every TCP connection).
- **ACK (Acknowledgement)** — Confirms that packets were received and indicates which sequence number the receiver expects next. This is how TCP guarantees reliable delivery.
- **RST (Reset)** — Raised when something goes wrong on the connection. When set, the connection is abruptly torn down rather than closed gracefully.
- **URG (Urgent)** — Marks a packet as high priority, telling the receiver to process it as soon as possible rather than waiting its turn in the buffer.
- **FIN (Finish)** — Indicates the sender has no more data to transmit and wants to close the connection — the polite, orderly way to end a session (versus RST's abrupt teardown).

- **PSH (Push)** — Forces buffered data to be delivered immediately instead of waiting for the buffer to fill, which helps at the start and end of transfers and avoids buffer deadlocks.

A couple of useful mental groupings: SYN and FIN bracket the lifecycle of a connection (open and close), ACK runs throughout to confirm receipt, and RST is the emergency abort. URG and PSH are both about timing/priority of data handling.



Host Discovery Techniques

Host discovery techniques — the methods a network scanner uses to figure out which machines on a network are alive and responding before doing any deeper probing. The subtitle frames it well: “How a network scanner probes a target to find which hosts are alive.

The following image is colour-coded into four tiers, shown in the legend at the bottom:

Objective (dark teal, top) — At the root sits “Discovering Hosts,” the overall goal everything branches from.

Scan methods (red, second tier) — These are the five main approaches, each based on a different network protocol:

- **ARP Ping Scan** — uses ARP requests; works only on the local network segment but is extremely reliable and fast there, since hosts can’t really hide from ARP.
- **UDP Ping Scan** — sends UDP packets to elicit a response (or an ICMP error that reveals the host exists).

- **ICMP Ping Scan** — the classic “ping” family, using ICMP messages.
- **IP Protocol Ping Scan** — sends packets using various IP protocols to see what the host responds to.
- **TCP Ping Scan** — uses TCP packets, useful when ICMP is blocked by firewalls.

Probe variants (blue, third tier) — Two of the scan methods break down into more specific probe types:

Under **ICMP Ping Scan**:

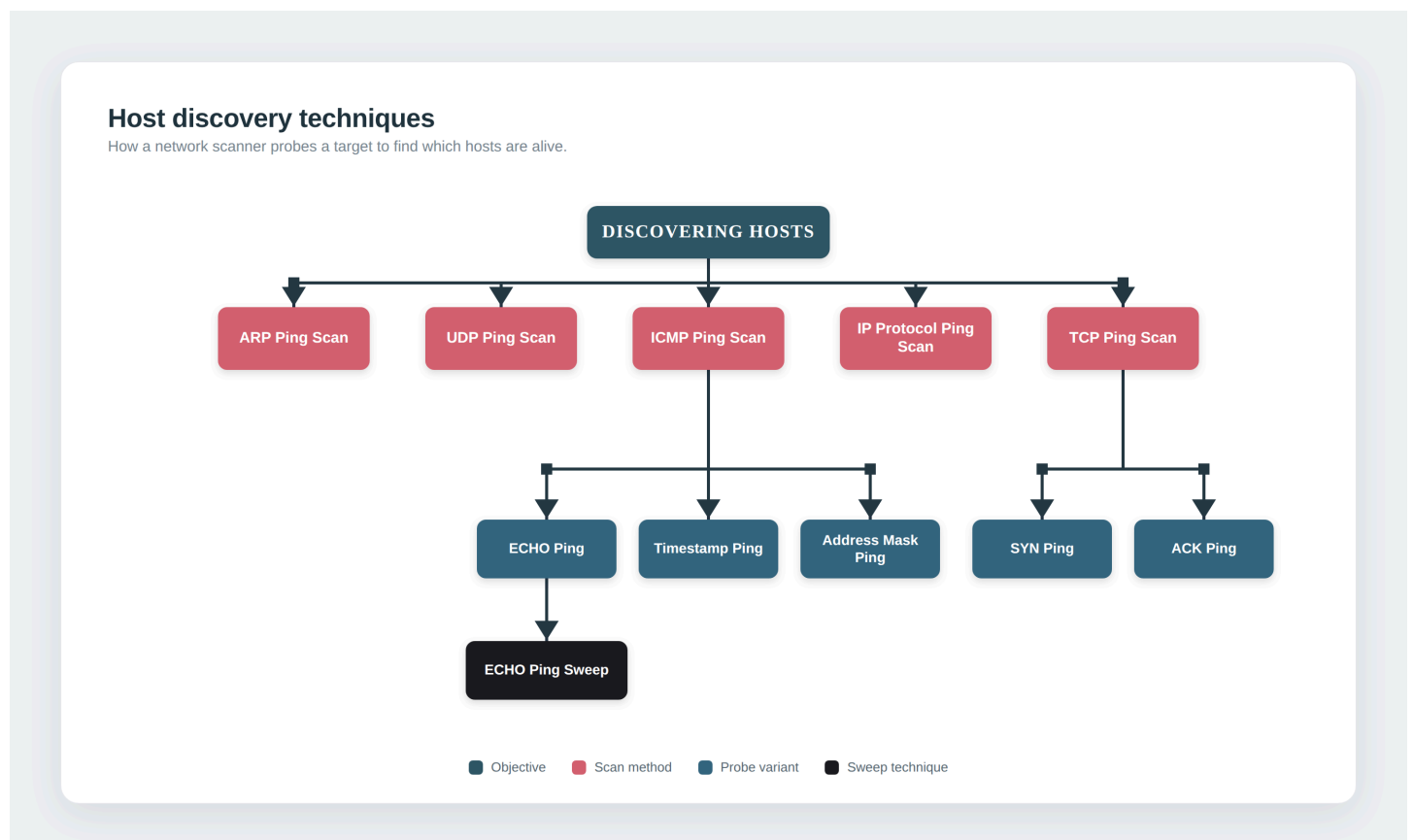
- **ECHO Ping** — the standard echo request/reply (what ping normally does)
- **Timestamp Ping** — asks the host for its timestamp
- **Address Mask Ping** — requests the host’s subnet mask

Under **TCP Ping Scan**:

- **SYN Ping** — sends a TCP SYN packet (ties back to the SYN flag from your earlier TCP question — it starts a handshake the host must respond to)
- **ACK Ping** — sends a TCP ACK packet, useful for slipping past stateless firewalls

Sweep technique (black, bottom) — **ECHO Ping Sweep** branches off ECHO Ping. A “sweep” means sending the same probe across a whole range of IP addresses at once rather than to a single host, to map out an entire subnet quickly.

The logic of the hierarchy is essentially: *goal* → *which protocol* → *which specific message type* → *applied across a range*. This is standard material in network reconnaissance; the kind of thing covered in tools like Nmap and in penetration-testing coursework.



Lab Environment

The attacker operates from Kali Linux at **192.168.1.17** and targets the host **192.168.1.9**, a VMware guest hardened from the pentest@ignite shell. The setup is summarised below.

- **Attacker:** Kali Linux — 192.168.1.17
- **Target:** 168.1.9 — VMware, MAC 00:0C:29:85:D1:03
- **Tooling:** Nmap 7.99, Wireshark, iptables
- **Convention:** every scan uses -sP with --disable-arp-ping so ARP cannot mask the result

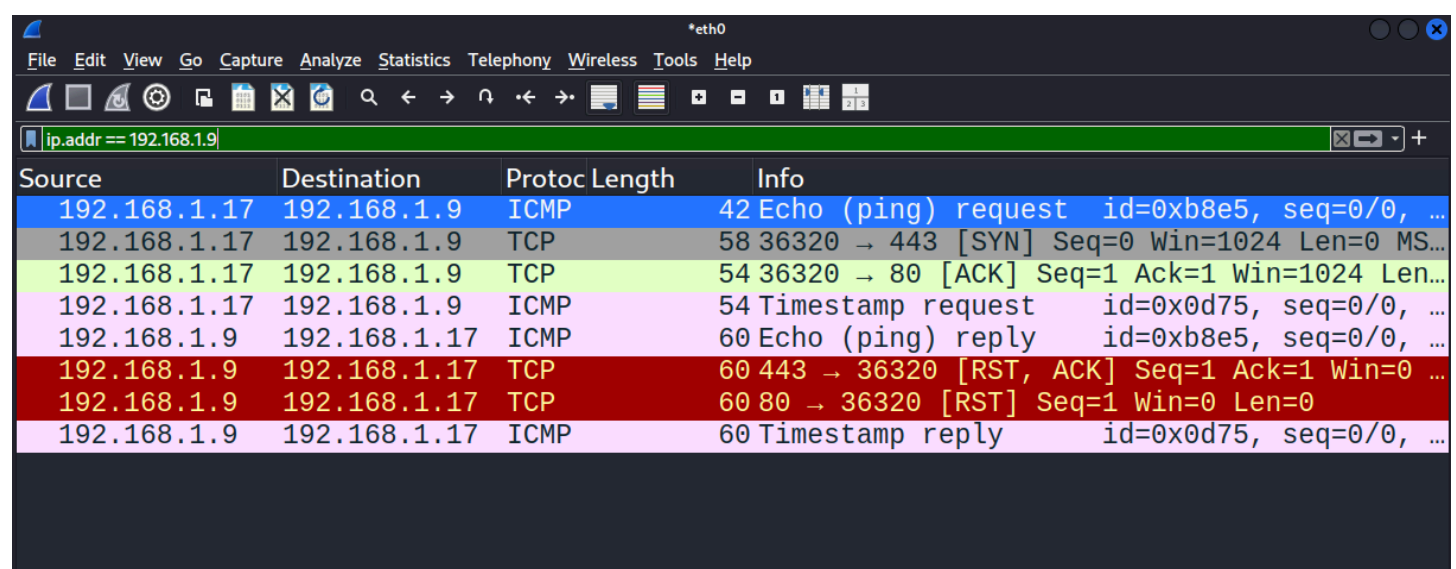
Establishing a Baseline: Default Host Discovery

With no firewall rules in place, the default ping scan succeeds immediately. Nmap reports the host as up with sub-millisecond latency, resolves its VMware MAC address, and completes the sweep in 0.60 seconds — a clean, unobstructed discovery.

```
1. nmap -sP 192.168.1.9 --disable-arp-ping
```

```
root@kali ~  
➤ nmap -sP 192.168.1.9 --disable-arp-ping  
Starting Nmap 7.99 ( https://nmap.org ) at 2026-05-26 14:41 -0400  
Nmap scan report for 192.168.1.9  
Host is up (0.00061s latency).  
MAC Address: 00:0C:29:85:D1:03 (VMware)  
Nmap done: 1 IP address (1 host up) scanned in 0.60 seconds  
root@kali ~
```

A simultaneous Wireshark capture reveals how Nmap reaches that verdict. Running with elevated privileges, it fires four parallel probes — an ICMP echo request, a TCP SYN to port 443, a TCP ACK to port 80, and an ICMP timestamp request — then accepts any single response as proof of life. The target obligingly returns an echo reply, TCP resets, and a timestamp reply.



The image shows a Wireshark packet capture on the eth0 interface. The filter is set to 'ip.addr == 192.168.1.9'. The capture shows a series of packets between 192.168.1.17 and 192.168.1.9. The packets include an ICMP echo request, a TCP SYN to port 443, a TCP ACK to port 80, and an ICMP timestamp request, followed by their respective responses: an ICMP echo reply, two TCP RST packets (one to port 443 and one to port 80), and an ICMP timestamp reply.

Source	Destination	Protocol	Length	Info
192.168.1.17	192.168.1.9	ICMP	42	Echo (ping) request id=0xb8e5, seq=0/0, ...
192.168.1.17	192.168.1.9	TCP	58	36320 → 443 [SYN] Seq=0 Win=1024 Len=0 MS...
192.168.1.17	192.168.1.9	TCP	54	36320 → 80 [ACK] Seq=1 Ack=1 Win=1024 Len...
192.168.1.17	192.168.1.9	ICMP	54	Timestamp request id=0x0d75, seq=0/0, ...
192.168.1.9	192.168.1.17	ICMP	60	Echo (ping) reply id=0xb8e5, seq=0/0, ...
192.168.1.9	192.168.1.17	TCP	60	443 → 36320 [RST, ACK] Seq=1 Ack=1 Win=0 ...
192.168.1.9	192.168.1.17	TCP	60	80 → 36320 [RST] Seq=1 Win=0 Len=0
192.168.1.9	192.168.1.17	ICMP	60	Timestamp reply id=0x0d75, seq=0/0, ...

Blocking the Default Probes

The defender now silences those four probes with three iptables rules: the first drops all ICMP traffic, the second drops the ACK probe destined for port 80, and the third drops the SYN probe destined for port 443. Each rule is inserted at the head of the INPUT chain, so it takes precedence over any later rule.

```

1. sudo iptables -I INPUT -p ICMP -j DROP
2. sudo iptables -I INPUT -p tcp --tcp-flags ALL ACK --dport 80 -j DROP
3. sudo iptables -I INPUT -p tcp --tcp-flags ALL SYN --dport 443 -j DROP

```

```

pentest@ignite:~$ sudo iptables -I INPUT -p ICMP -j DROP
pentest@ignite:~$
pentest@ignite:~$ sudo iptables -I INPUT -p tcp --tcp-flags ALL ACK --dport 80 -j DROP
pentest@ignite:~$
pentest@ignite:~$ sudo iptables -I INPUT -p tcp --tcp-flags ALL SYN --dport 443 -j DROP
pentest@ignite:~$

```

Re-running the identical scan now fails. Nmap receives no replies, declares “0 hosts up,” and suggests -Pn for situations where a live host blocks probes. The scan also stretches to 3.05 seconds — roughly five times longer — because Nmap waits and retransmits before conceding.

```

1. nmap -sP 192.168.1.9 --disable-arp-ping

```

```

root@kali ~
└─$ nmap -sP 192.168.1.9 --disable-arp-ping
Starting Nmap 7.99 ( https://nmap.org ) at 2026-05-26 14:45 -0400
Note: Host seems down. If it is really up, but blocking our ping probes, try -Pn
Nmap done: 1 IP address (0 hosts up) scanned in 3.05 seconds
root@kali ~

```

The capture confirms the cause. Nmap transmits its full probe set, and even retransmits, but the target returns nothing. The annotation “NO Reply Packet” marks the silence the firewall now enforces.

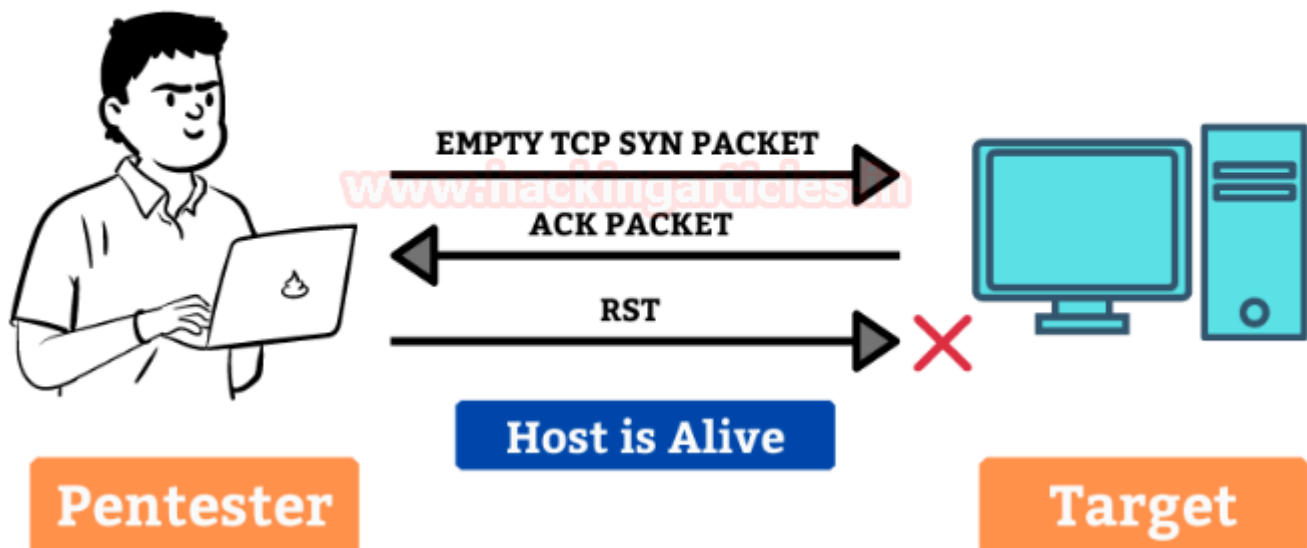
Source	Destination	Protoc	Length	Info
192.168.1.17	192.168.1.9	ICMP	42	Echo (ping) request id=0x17dc, seq=0/0, ...
192.168.1.17	192.168.1.9	TCP	58	49032 → 443 [SYN] Seq=0 Win=1024 Len=0 MS...
192.168.1.17	192.168.1.9	TCP	54	49032 → 80 [ACK] Seq=1 Ack=1 Win=1024 Len...
192.168.1.17	192.168.1.9	ICMP	54	Timestamp request id=0xe6ee, seq=0/0, ...
192.168.1.17	192.168.1.9	ICMP	54	Timestamp request id=0x1bbe, seq=0/0, ...
192.168.1.17	192.168.1.9	TCP	54	49034 → 80 [ACK] Seq=1 Ack=1 Win=1024 Len...
192.168.1.17	192.168.1.9	TCP	58	49034 → 443 [SYN] Seq=0 Win=1024 Len=0 MS...
192.168.1.17	192.168.1.9	ICMP	42	Echo (ping) request id=0xcd14, seq=0/0, ...

NO Reply Packet

TCP SYN Ping (-PS)

The following image shows the TCP SYN Ping Scan diagram, which shows a three-step exchange. The Pentester sends an empty TCP SYN packet to the Target, pretending to open a connection. If the host is alive, it replies with an ACK packet (technically SYN-ACK), confirming it received the probe. The Pentester then immediately sends an RST to abruptly tear down the half-open connection rather than completing it. That returned reply proves the Host is Alive. This technique works even when ICMP pings are blocked.

TCP SYN PING SCAN



To defeat the block, the attacker selects a specific probe. The TCP SYN ping sends a bare SYN segment, by default to port 80 — a port the earlier rules left open to SYN traffic. Discovery succeeds again, reporting one host up in 0.62 seconds.

```
1. nmap -sP -PS 192.168.1.9 --disable-arp-ping
```

```
root@kali ~  
-> nmap -sP -PS 192.168.1.9 --disable-arp-ping  
Starting Nmap 7.99 ( https://nmap.org ) at 2026-05-26 14:51 -0400  
Nmap scan report for 192.168.1.9  
Host is up (0.00074s latency).  
MAC Address: 00:0C:29:85:D1:03 (VMware)  
Nmap done: 1 IP address (1 host up) scanned in 0.62 seconds  
root@kali ~
```

Wireshark shows the classic half-open exchange. Nmap sends a SYN to port 80, the target answers with SYN/ACK — implying both a reachable port and a live host — and Nmap immediately tears the nascent connection down with a RST.

Source	Destination	Protoc	Length	Info
192.168.1.17	192.168.1.9	TCP	58	33058 → 80 [SYN] Seq=0 Win=1024 Len=0 MSS...
192.168.1.9	192.168.1.17	TCP	60	80 → 33058 [SYN, ACK] Seq=0 Ack=1 Win=642...
192.168.1.17	192.168.1.9	TCP	54	33058 → 80 [RST] Seq=1 Win=0 Len=0

The defender responds by dropping every inbound TCP segment that carries only the SYN flag, irrespective of destination port.

```
1. sudo iptables -I INPUT -p tcp --tcp-flags ALL SYN -j DROP
```

```
pentest@ignite:~$  
pentest@ignite:~$ sudo iptables -I INPUT -p tcp --tcp-flags ALL SYN -j DROP  
pentest@ignite:~$
```

With SYN probes discarded, the SYN ping fails as well — 0 hosts up, and once more the -Pn hint.

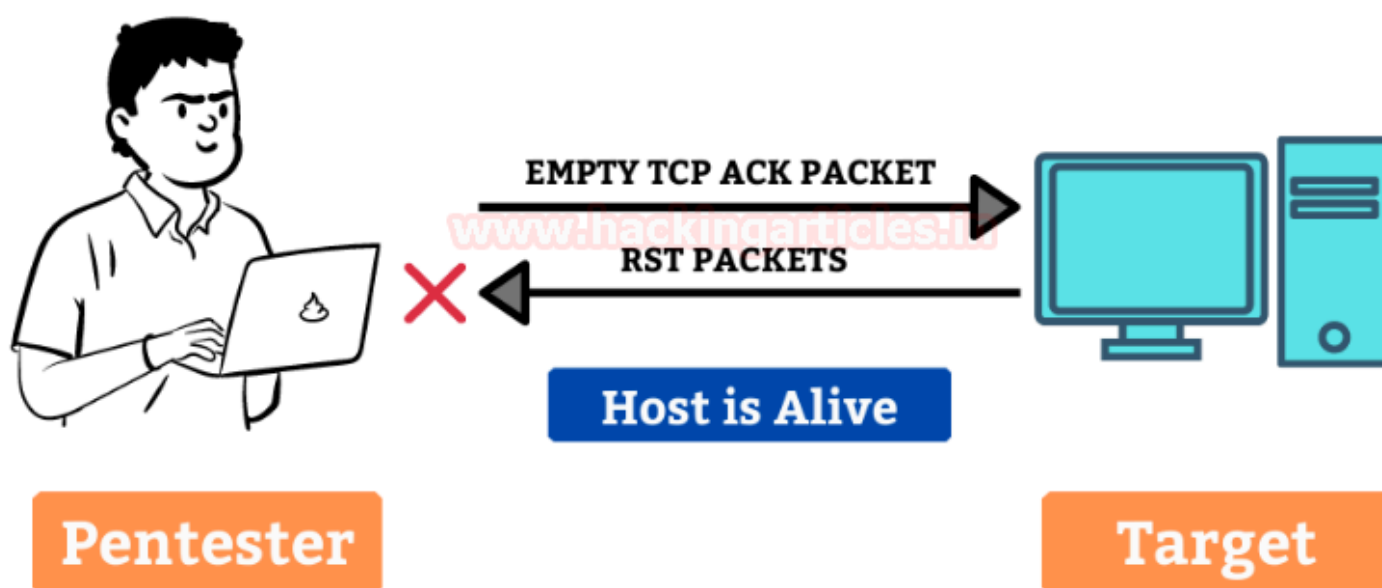
```
1. nmap -sP -PS 192.168.1.9 --disable-arp-ping
```

```
root@kali ~  
➤ nmap -sP -PS 192.168.1.9 --disable-arp-ping  
Starting Nmap 7.99 ( https://nmap.org ) at 2026-05-26 14:54 -0400  
Note: Host seems down. If it is really up, but blocking our ping probes, try -Pn  
Nmap done: 1 IP address (0 hosts up) scanned in 2.05 seconds  
root@kali ~
```

TCP ACK Ping (-PA)

The TCP ACK Ping Scan image below shows a two-step exchange. The Pentester sends an empty TCP ACK packet to the Target, pretending to acknowledge a connection that was never opened. Since no real connection exists, a live host responds with an RST packet to reject the unexpected acknowledgement. That returned RST proves the Host is Alive. The red cross suggests firewalls may filter the reply, but this technique is useful for slipping past stateless packet-filtering defences.

TCP ACK PING SCAN

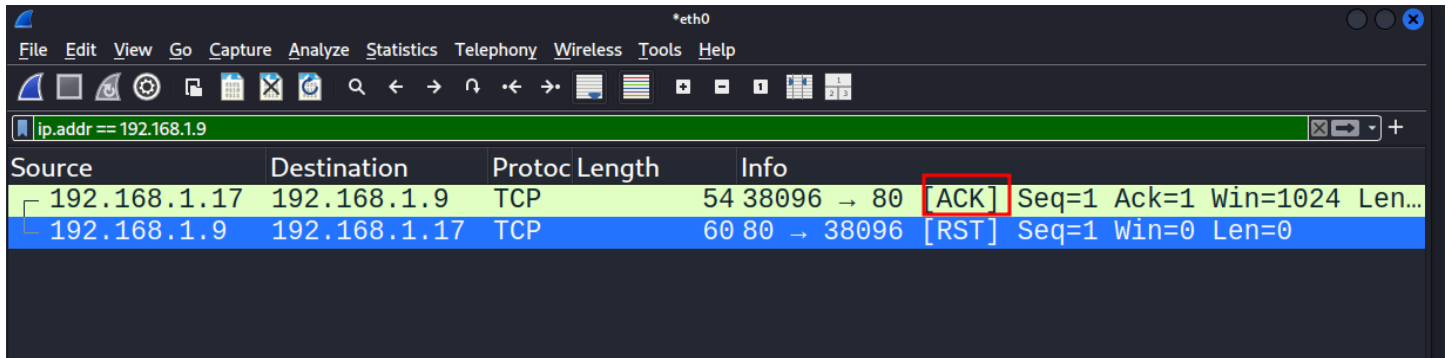


The TCP ACK ping takes a different tack, sending a lone ACK segment. Because no connection exists to acknowledge, a live host must answer with a RST — and that RST itself proves liveness. The scan duly reports one host up.

```
1. nmap -sP -PA 192.168.1.9 --disable-arp-ping
```

```
root@kali ~  
➤ nmap -sP -PA 192.168.1.9 --disable-arp-ping  
Starting Nmap 7.99 ( https://nmap.org ) at 2026-05-26 14:55 -0400  
Nmap scan report for 192.168.1.9  
Host is up (0.00056s latency).  
MAC Address: 00:0C:29:85:D1:03 (VMware)  
Nmap done: 1 IP address (1 host up) scanned in 0.62 seconds  
root@kali ~
```


The capture is minimal but decisive: Nmap sends an ACK to port 80, the target returns a RST, and discovery completes.



Source	Destination	Protoc	Length	Info
192.168.1.17	192.168.1.9	TCP	54	38096 → 80 [ACK] Seq=1 Ack=1 Win=1024 Len...
192.168.1.9	192.168.1.17	TCP	60	80 → 38096 [RST] Seq=1 Win=0 Len=0

The defender now drops inbound TCP segments that bear only the ACK flag.

```
1. sudo iptables -I INPUT -p tcp --tcp-flags ALL ACK -j DROP
```

```
pentest@ignite:~$  
pentest@ignite:~$ sudo iptables -I INPUT -p tcp --tcp-flags ALL ACK -j DROP  
pentest@ignite:~$
```

The ACK ping collapses accordingly, reporting 0 hosts up.

```
1. nmap -sP -PA 192.168.1.9 --disable-arp-ping
```

```
root@kali ~  
➤ nmap -sP -PA 192.168.1.9 --disable-arp-ping  
Starting Nmap 7.99 ( https://nmap.org ) at 2026-05-26 14:56 -0400  
Note: Host seems down. If it is really up, but blocking our ping probes, try -Pn  
Nmap done: 1 IP address (0 hosts up) scanned in 2.06 seconds  
root@kali ~
```

ICMP Echo Ping (-PE)

The ICMP Echo Ping Scan image below shows the classic ping exchange. The Pentester sends an ICMP Echo Request to the Target across the network, represented by the globe icon. If the host is alive and not blocking ICMP, it returns an ICMP Echo Reply. That reply confirms the target is online. This is exactly what the standard ping command does. It works across networks, unlike ARP, but firewalls frequently block ICMP traffic, which can cause live hosts to appear unreachable.

ICMP ECHO PING SCAN

www.hackingarticles.in



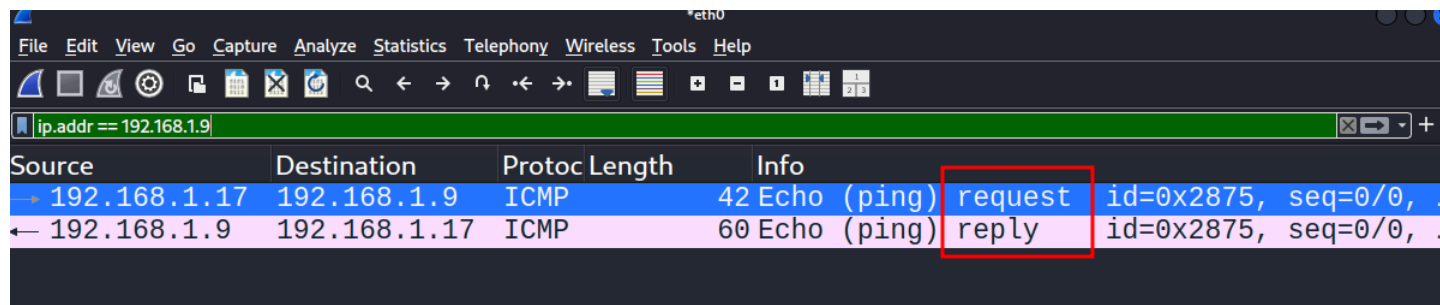
The ICMP echo ping issues a textbook ping — an ICMP echo request — and treats the echo reply as confirmation.

```
1. nmap -sP -PE 192.168.1.9 --disable-arp-ping
```

```
root@kali ~  
➔ nmap -sP -PE 192.168.1.9 --disable-arp-ping  
Starting Nmap 7.99 ( https://nmap.org ) at 2026-05-26 14:58 -0400  
Nmap scan report for 192.168.1.9  
Host is up (0.00055s latency).  
MAC Address: 00:0C:29:85:D1:03 (VMware)  
Nmap done: 1 IP address (1 host up) scanned in 0.61 seconds  
root@kali ~
```

The host answers and is reported up.

Wireshark captures the pair plainly: an echo request leaves the attacker, an echo reply to returns, and both share the same ICMP identifier.



A targeted rule now drops only ICMP echo-request packets, blinding the echo ping while leaving other ICMP types untouched.

```
1. sudo iptables -A INPUT -p icmp --icmp-type echo-request -j DROP
```

```
pentest@ignite:~$  
pentest@ignite:~$ sudo iptables -A INPUT -p icmp --icmp-type echo-request -j DROP  
pentest@ignite:~$
```

The echo ping fails — 0 hosts up — confirming the rule's effect.

```
1. nmap -sP -PE 192.168.1.9 --disable-arp-ping
```

```
root@kali ~  
➔ nmap -sP -PE 192.168.1.9 --disable-arp-ping  
Starting Nmap 7.99 ( https://nmap.org ) at 2026-05-26 15:00 -0400  
Note: Host seems down. If it is really up, but blocking our ping probes, try -Pn  
Nmap done: 1 IP address (0 hosts up) scanned in 2.05 seconds  
root@kali ~
```

ICMP Timestamp Ping (-PP)

When echo requests are blocked, the ICMP timestamp ping offers an alternative. It sends an ICMP timestamp request — a type defender often overlooks — and the target replies, reporting the host up in 0.61 seconds.

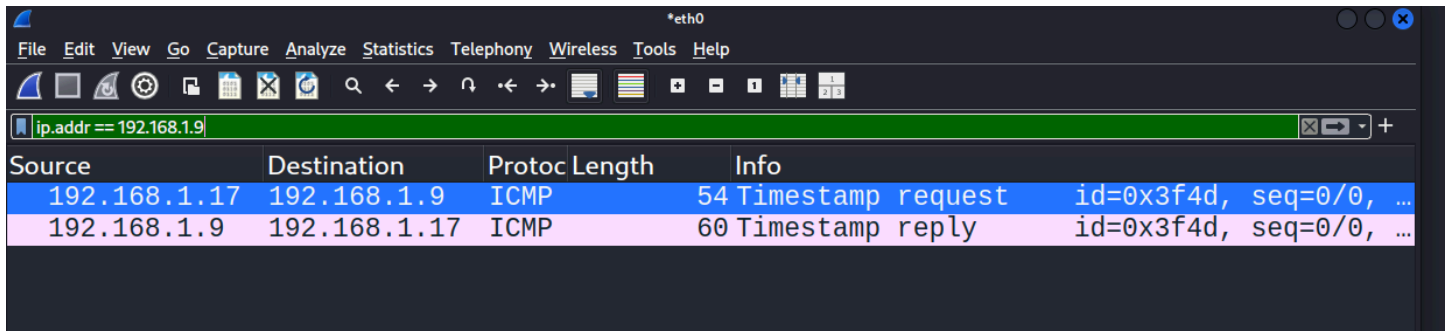
```
1. nmap -sP -PP 192.168.1.9 --disable-arp-ping
```

```

root@kali ~
└─> nmap -sP -PP 192.168.1.9 --disable-arp-ping
Starting Nmap 7.99 ( https://nmap.org ) at 2026-05-26 15:01 -0400
Nmap scan report for 192.168.1.9
Host is up (0.00047s latency).
MAC Address: 00:0C:29:85:D1:03 (VMware)
Nmap done: 1 IP address (1 host up) scanned in 0.61 seconds
root@kali ~

```

The capture shows the timestamp request and its matching reply, again sharing a single identifier.



Source	Destination	Protoc	Length	Info
192.168.1.17	192.168.1.9	ICMP	54	Timestamp request id=0x3f4d, seq=0/0, ...
192.168.1.9	192.168.1.17	ICMP	60	Timestamp reply id=0x3f4d, seq=0/0, ...

To close this gap, the defender drops ICMP wholesale. A single rule against all ICMP traffic removes both the echo and the timestamp avenues at once — and, as the next section confirms, defeats the timestamp ping when it is retried.

```
1. sudo iptables -I INPUT -p ICMP -j DROP
```

```

pentest@ignite:~$ sudo iptables -I INPUT -p ICMP -j DROP
pentest@ignite:~$

```

With ICMP dropped in its entirety, re-issuing the timestamp ping now returns nothing useful: Nmap reports 0 hosts up and repeats its -Pn suggestion. Every ICMP-based probe is blocked, so the attacker must abandon ICMP and look elsewhere.

```
1. nmap -sP -PP 192.168.1.9 --disable-arp-ping
```

```

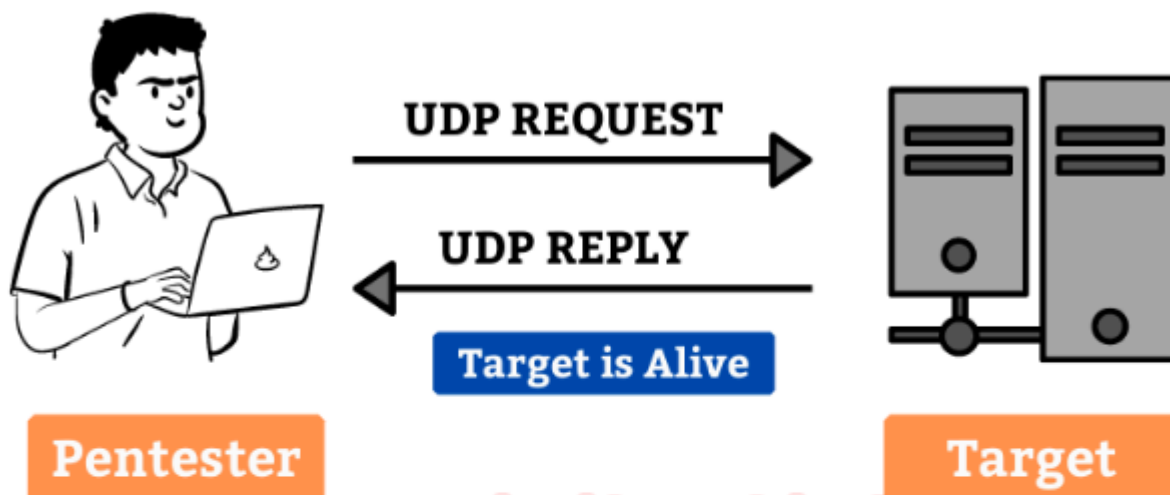
root@kali ~
└─> nmap -sP -PP 192.168.1.9 --disable-arp-ping
Starting Nmap 7.99 ( https://nmap.org ) at 2026-05-26 15:03 -0400
Note: Host seems down. If it is really up, but blocking our ping probes, try -Pn
Nmap done: 1 IP address (0 hosts up) scanned in 2.05 seconds
root@kali ~

```

UDP Ping (-PU)

The UDP Ping Scan image below shows two outcomes. In the top panel, the Pentester sends a UDP Request, and the Target returns a UDP Reply, confirming the host is alive. In the bottom panel, the same request triggers a TTL Exceeded or destination-unreachable error instead, signalling the target is dead or filtered. UDP scanning is interpreted differently from TCP: a reply or specific error reveals the host's state. It helps probe services that ignore TCP or ICMP traffic entirely.

UDP PING SCAN WHEN TARGET IS ACTIVE



www.hackingarticles.in

UDP PING SCAN WHEN TARGET IS INACTIVE



The UDP ping sends a datagram to a typically closed port and waits for the telltale reply. Against the host, it works at once — Nmap resolves the MAC address and reports one host up in 0.68 seconds.

```
1. nmap -sP -PU 192.168.1.9 --disable-arp-ping
```

```
root@kali ~  
➤ nmap -sP -PU 192.168.1.9 --disable-arp-ping  
Starting Nmap 7.99 ( https://nmap.org ) at 2026-05-26 15:03 -0400  
Nmap scan report for 192.168.1.9  
Host is up (0.00045s latency).  
MAC Address: 00:0C:29:85:D1:03 (VMware)  
Nmap done: 1 IP address (1 host up) scanned in 0.68 seconds  
root@kali ~
```

Wireshark explains the mechanism. Nmap sends a small UDP datagram to a high port — here 40125 — and because nothing listens there, the target's own stack answers with an ICMP "Destination unreachable (Port unreachable)" message. That error is itself proof that the host is alive.

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

ip.addr == 192.168.1.9

Source	Destination	Protoc	Length	Info
192.168.1.17	192.168.1.9	UDP	82	56133 → 40125 Len=40
192.168.1.9	192.168.1.17	ICMP	110	Destination unreachable (Port unreachable)

The defender closes the vector by dropping all inbound UDP.

```
1. sudo iptables -I INPUT -p UDP -j DROP
```

```
pentest@ignite:~$
pentest@ignite:~$ sudo iptables -I INPUT -p UDP -j DROP
pentest@ignite:~$
```

Deprived of the port-unreachable reply, the UDP ping fails — 0 hosts up.

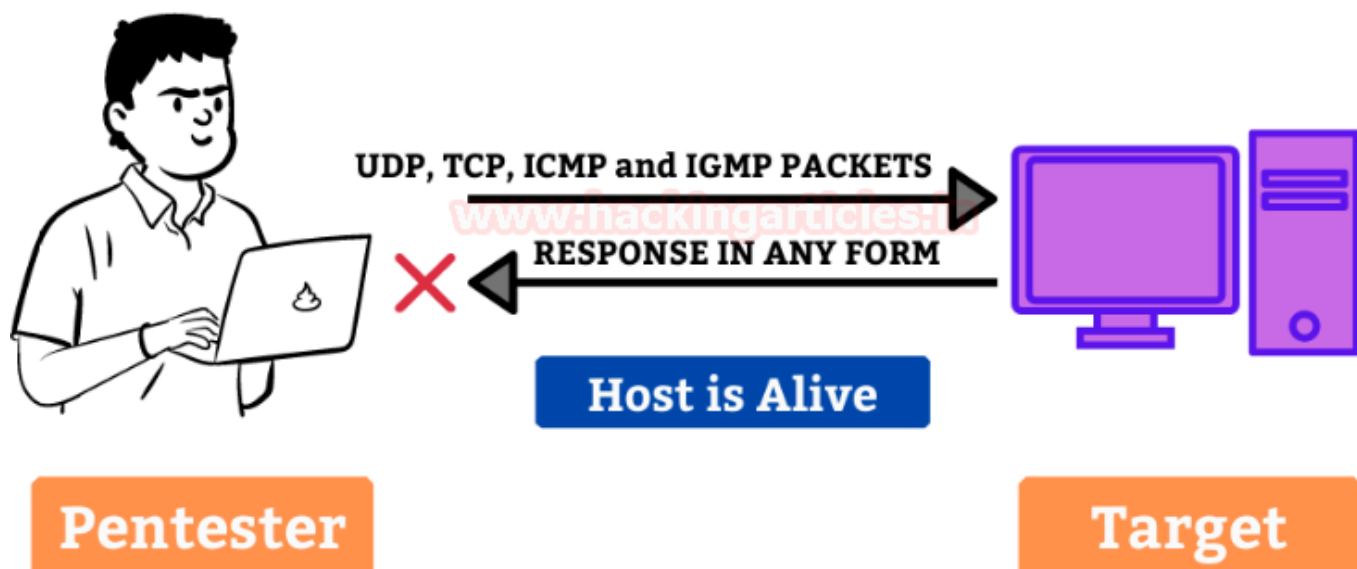
```
1. nmap -sP -PU 192.168.1.9 --disable-arp-ping
```

```
root@kali ~
> nmap -sP -PU 192.168.1.9 --disable-arp-ping
Starting Nmap 7.99 ( https://nmap.org ) at 2026-05-26 15:05 -0400
Note: Host seems down. If it is really up, but blocking our ping probes, try -Pn
Nmap done: 1 IP address (0 hosts up) scanned in 2.12 seconds
root@kali ~
```

IP-Protocol Ping (-PO)

The IP Protocol Ping Scan diagram shows a broad probing approach. The Pentester sends multiple packet types at once, including UDP, TCP, ICMP and IGMP, to the Target. Rather than expecting one specific answer, the scanner accepts a response in any form. If the host replies with anything at all, it proves the Host is Alive. The red cross indicates some replies may be filtered. This shotgun method increases discovery odds by testing several protocols simultaneously, catching hosts that selectively block traffic.

IP PROTOCOL PING SCAN

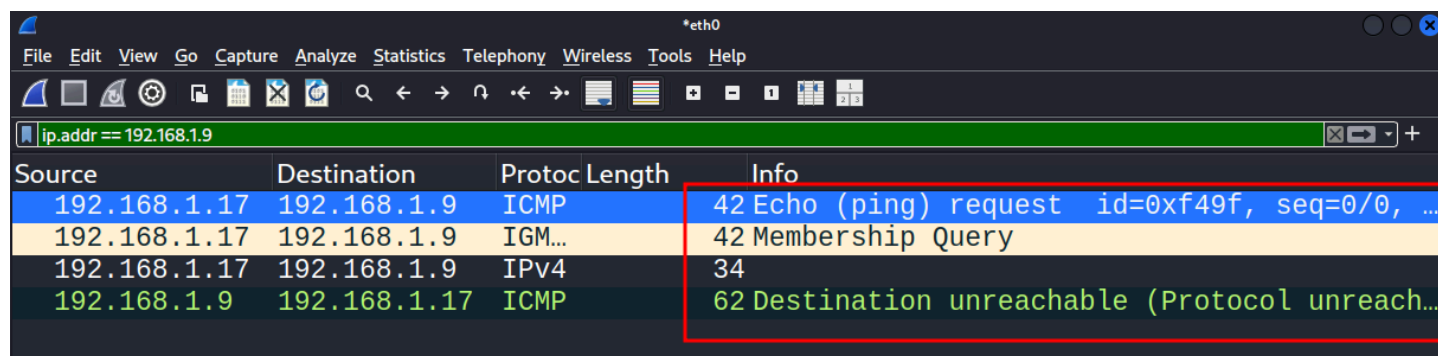


The IP protocol ping probes liveness at the network layer. It sends packets carrying several different IP protocol numbers and accepts any response — or any protocol-unreachable error — as confirmation. The host answers, and Nmap reports it up in 0.61 seconds.

```
1. nmap -sP -PO 192.168.1.9 --disable-arp-ping
```

```
root@kali ~  
➤ nmap -sP -PO 192.168.1.9 --disable-arp-ping  
Starting Nmap 7.99 ( https://nmap.org ) at 2026-05-26 15:07 -0400  
Nmap scan report for 192.168.1.9  
Host is up (0.00041s latency).  
MAC Address: 00:0C:29:85:D1:03 (VMware)  
Nmap done: 1 IP address (1 host up) scanned in 0.61 seconds  
root@kali ~
```

The capture shows the probe set clearly. Nmap dispatches packets across multiple protocols — an ICMP echo request, an IGMP membership query, and a raw IPv4 (IP-in-IP) packet — and the target replies with an ICMP “Destination unreachable (Protocol unreachable),” once again betraying its presence.



A Wireshark packet capture window titled '*eth0' showing traffic on interface 'ip.addr == 192.168.1.9'. The packet list table is as follows:

Source	Destination	Protoc	Length	Info
192.168.1.17	192.168.1.9	ICMP	42	Echo (ping) request id=0xf49f, seq=0/0, ...
192.168.1.17	192.168.1.9	IGMP	42	Membership Query
192.168.1.17	192.168.1.9	IPv4	34	
192.168.1.9	192.168.1.17	ICMP	62	Destination unreachable (Protocol unreach...

The defender adds a rule dropping the IP-protocol traffic that the probe depends on.

```
1. sudo iptables -I INPUT -p IP -j DROP
```

```
pentest@ignite:~$  
pentest@ignite:~$ sudo iptables -I INPUT -p IP -j DROP  
pentest@ignite:~$
```

The IP-protocol ping now fails as well, reporting 0 hosts up after a 3.06-second wait.

```
1. nmap -sP -PO 192.168.1.9 --disable-arp-ping
```

```
root@kali ~  
➤ nmap -sP -PO 192.168.1.9 --disable-arp-ping  
Starting Nmap 7.99 ( https://nmap.org ) at 2026-05-26 15:09 -0400  
Note: Host seems down. If it is really up, but blocking our ping probes, try -Pn  
Nmap done: 1 IP address (0 hosts up) scanned in 3.06 seconds  
root@kali ~
```

ARP Ping (-PR)

On a directly connected Layer 2 network, ARP ping is Nmap’s most powerful and dependable discovery method. Because ARP works beneath the IP layer, it bypasses iptables entirely: no INPUT-chain rule can catch or drop an ARP frame. Consequently, the probe avoids the firewall

techniques described earlier. With the -PR option, Nmap broadcasts an ARP “Who has 192.168.1.9?” request; the target’s ARP implementation must reply with its MAC address.

ARP PING SCAN



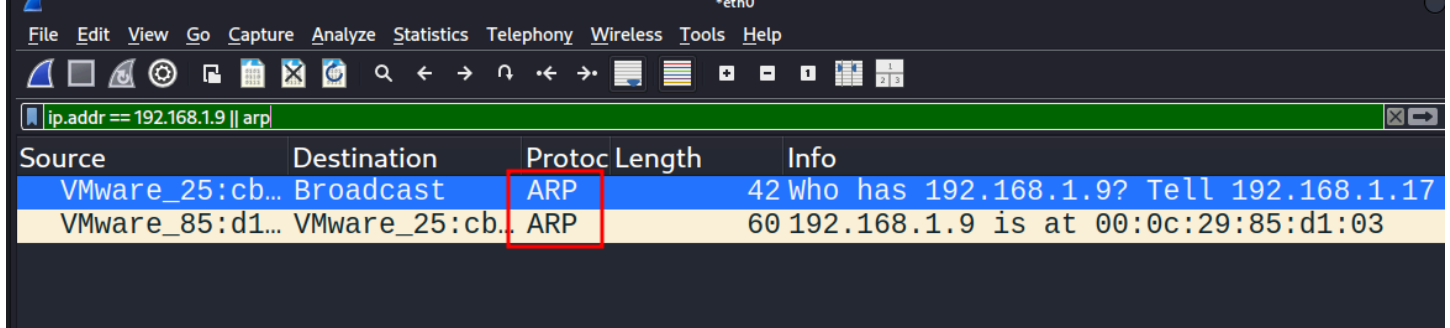
The Nmap command for this scan will identify the VMware MAC vendor prefix, verify the host is up, and complete the scan in just 0.57 seconds.

```
1. nmap -sP -PR 192.168.1.9
```

```
root@kali ~  
└─> nmap -sP -PR 192.168.1.9  
Starting Nmap 7.99 ( https://nmap.org ) at 2026-05-28 18:49 -0400  
Nmap scan report for 192.168.1.9  
Host is up (0.00035s latency).  
MAC Address: 00:0C:29:85:D1:03 (VMware)  
Nmap done: 1 IP address (1 host up) scanned in 0.57 seconds  
root@kali ~
```

Nmap shows “Host is up (0.00035s latency)” and identifies the MAC as a VMware NIC—a handy virtualisation fingerprint. The sub-millisecond RTT reflects a pure Layer-2 exchange: no routing or IP processing, just a broadcast and direct reply.

A concurrent Wireshark capture confirms the frame-level exchange. The filter `ip.addr == 192.168.1.9 || arp` shows two frames: an ARP broadcast from the attacker’s VMware NIC asking “Who has 192.168.1.9? Tell 192.168.1.17,” and the target’s unicast reply “192.168.1.9 is at 00:0c:29:85:d1:03.” No higher-layer protocols are involved; the discovery completes in two 42- and 60-byte frames. Since iptables inspects IP headers, and ARP frames lack them, the firewall cannot see or block this traffic.

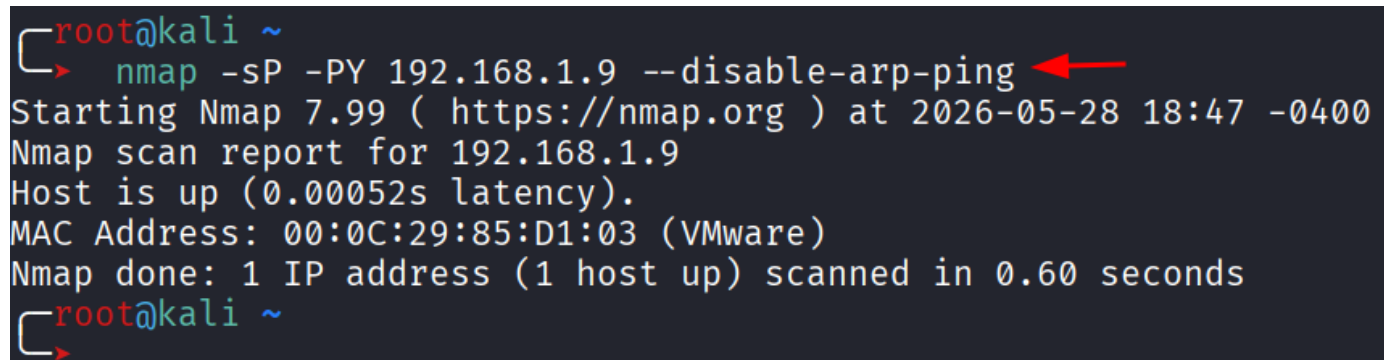


Source	Destination	Protoc	Length	Info
VMware_25:cb...	Broadcast	ARP	42	Who has 192.168.1.9? Tell 192.168.1.17
VMware_85:d1...	VMware_25:cb...	ARP	60	192.168.1.9 is at 00:0c:29:85:d1:03

SCTP INIT Ping (-PY)

The SCTP INIT ping uses Stream Control Transmission Protocol — a transport-layer alternative to TCP/UDP — to detect hosts via a path many firewalls ignore. Instead of TCP’s SYN/ACK or RST, it sends an SCTP INIT chunk and treats an INIT-ACK (open) or an ABORT (closed) as proof of life. Even an ICMP “Protocol unreachable” (no SCTP support) counts as liveness, so the method works against hosts without SCTP services. With ARP suppressed, the scan here reported one host up in 0.60 seconds and resolved the VMware MAC.

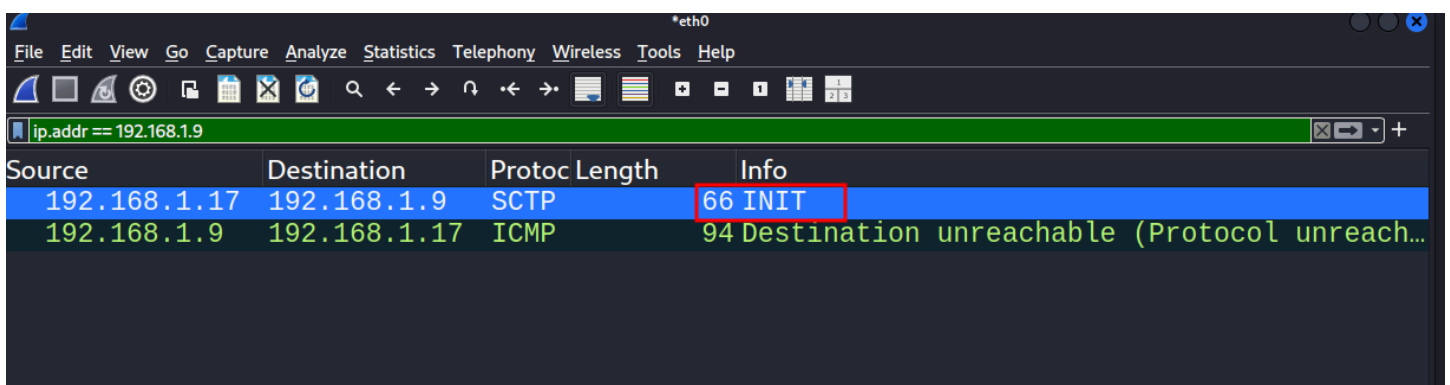
```
1. nmap -sP -PY 192.168.1.9 --disable-arp-ping
```



```
root@kali ~
> nmap -sP -PY 192.168.1.9 --disable-arp-ping
Starting Nmap 7.99 ( https://nmap.org ) at 2026-05-28 18:47 -0400
Nmap scan report for 192.168.1.9
Host is up (0.00052s latency).
MAC Address: 00:0C:29:85:D1:03 (VMware)
Nmap done: 1 IP address (1 host up) scanned in 0.60 seconds
root@kali ~
```

The terminal shows the same pattern: sub-millisecond latency, MAC resolution, and one host up. The difference is procedural: the target lacks SCTP, so the kernel has no socket for the INIT chunk and must emit an error reply—thereby revealing the host.

The Wireshark trace clarifies the exchange: the attacker sends a 66-byte SCTP INIT (like a TCP SYN) from 192.168.1.17 to 192.168.1.9. With no SCTP listener, the target’s kernel returns a 94-byte ICMP “Destination unreachable (Protocol unreachable)” — Nmap treats that error as proof the host is alive. A filtered or offline host would remain silent.



Source	Destination	Protoc	Length	Info
192.168.1.17	192.168.1.9	SCTP	66	INIT
192.168.1.9	192.168.1.17	ICMP	94	Destination unreachable (Protocol unreach...

For defenders, this is important: SCTP is rarely covered by explicit firewall rules, so an otherwise strict policy can still leak liveness via ICMP protocol-unreachable. Mitigation requires dropping inbound SCTP or using a default-DROP posture that denies any protocol not explicitly allowed.

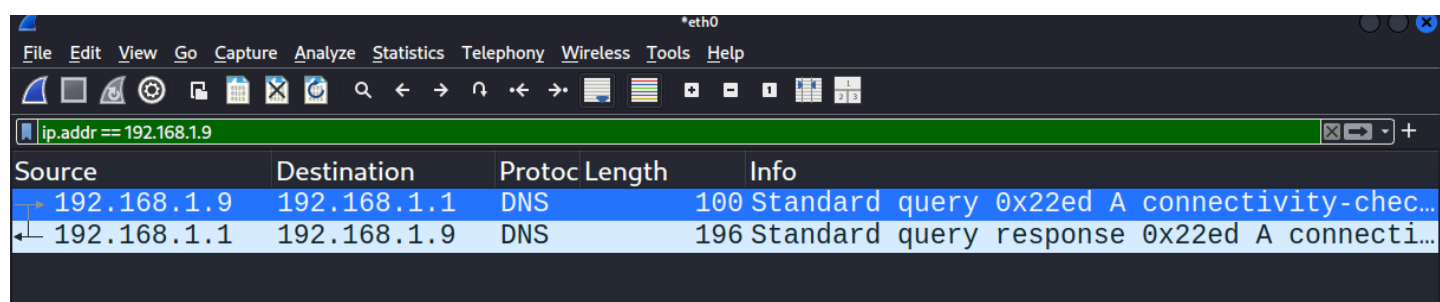
When Every Probe Fails: Skip Discovery (-Pn)

With each discovery method neutralised, the attacker stops asking the question. The -Pn option — surfaced repeatedly in Nmap's earlier hints — disables host discovery entirely. Nmap simply assumes the target is up and prepares to scan it directly, reporting "Host is up" in 0.50 seconds without sending a single discovery probe.

```
1. nmap -sP -PN 192.168.1.9 --disable-arp-ping
```

```
root@kali ~  
➤ nmap -sP -PN 192.168.1.9 --disable-arp-ping  
Starting Nmap 7.99 ( https://nmap.org ) at 2026-05-26 15:10 -0400  
Nmap scan report for 192.168.1.9  
Host is up.  
Nmap done: 1 IP address (1 host up) scanned in 0.50 seconds  
root@kali ~
```

The capture proves the point. Filtered by the target's address, it contains no attacker-originated probes at all; the only packets are the host's own routine DNS connectivity checks to the gateway. Nmap transmitted nothing — it merely declared the host up and moved on.



The image shows a Wireshark packet capture on the eth0 interface, filtered by ip.addr == 192.168.1.9. The capture shows two DNS packets: a standard query from 192.168.1.9 to 192.168.1.1, and a standard query response from 192.168.1.1 to 192.168.1.9. No Nmap probes are visible.

Source	Destination	Protoc	Length	Info
192.168.1.9	192.168.1.1	DNS	100	Standard query 0x22ed A connectivity-chec...
192.168.1.1	192.168.1.9	DNS	196	Standard query response 0x22ed A connecti...

Mitigation Strategies

The experiment makes one defensive principle unmistakable: silencing a single probe merely redirects the attacker to the next, so a host only disappears from a probe-based sweep when every discovery vector is filtered together. The following measures, drawn directly from the techniques demonstrated above, form a layered approach to suppressing host discovery.

- **Filter every discovery vector, not one.** Combine rules for ICMP, TCP SYN, TCP ACK, UDP, and the IP-protocol probes; a gap in any one of them keeps the host visible, exactly as the SYN, ACK, UDP, and timestamp pings each demonstrated.
- **Drop ICMP comprehensively.** Filtering only echo-request still leaves the timestamp ping working; a single rule against all ICMP closes echo, timestamp, and the unreachable replies that UDP and IP-protocol probes exploit.
- **Default to a DROP policy.** Prefer an explicit default-deny INPUT policy with a narrow allow-list over a long chain of per-probe DROP rules, so unanticipated probe types are refused by default rather than slipping through.
- **Prefer DROP over REJECT.** DROP returns nothing, whereas REJECT emits an ICMP error that itself confirms liveness — the very behaviour the UDP and IP-protocol pings rely on.
- **Suppress unreachable messages.** Rate-limit or block outbound ICMP port-unreachable and protocol-unreachable responses, removing the error replies that betray a closed port or

unsupported protocol.

- **Insert rules at the head of the chain.** Use -I so deny rules take precedence over broad accept rules already present, ensuring probes are dropped before any permissive rule matches them.
- **Monitor for the residual signal.** A determined attacker who exhausts discovery falls back to -Pn and scans blind; watch for the high-volume, reply-less probe traffic that pattern produces, since concealment hides the host but not the scan.
- **Validate from the attacker's view.** After applying rules, re-run each Nmap probe type, as shown throughout this article, to confirm every vector returns 0 hosts up before trusting the configuration.

Conclusion

This article climbed the entire ladder of Nmap host discovery — ARP, ICMP echo and timestamp, TCP SYN and ACK, UDP, and IP-protocol probes — and watched a determined defender knock out each rung with a focused iptables rule. The exercise yields a clear, two-sided lesson. For the defender, no single rule suffices; only by filtering every discovery vector does a host truly disappear from a probe-based sweep. For the attacker, that very completeness is the cue to change tactics: when -Pn becomes the only path forward, discovery ends and direct scanning begins. Liveness, in the end, is not a question the network must answer — it is an assumption the attacker can simply choose to make, which is precisely why robust defence aims not to win the discovery exchange outright but to deny the attacker any reliable signal at all.



FACEBOOK



TWITTER



PINTEREST



LINKEDIN

◀ PREVIOUS POST

Comprehensive Guide on Autopsy Tool (Windows)

NEXT POST ▶

Android Hooking and SSLPinning using Objection Framework

5 thoughts on “Nmap for Pentester: Host Discovery”



John

December 16, 2020 at 12:25 pm

Most websites now have defence against NMap so its no good for some sites.

Reply



Prashant Kadam

December 17, 2020 at 2:56 pm

Very good article

Reply



Muhammad usman

December 21, 2020 at 1:00 pm

kindly suggests me youtube courses to become kali Linux expert for penetration,hacking ,network security etc.

Reply

Palashtaru Dasgupta

March 25, 2021 at 6:48 am

nicely explained

Reply

Anonymous

October 27, 2024 at 4:51 pm

fantastic

Reply

Leave a Reply

Your email address will not be published. Required fields are marked *

Comment * *

Name *

Email *

Website

☐ Save my name, email, and website in this browser for the next time I comment.

Post Comment

Categories

Select Category



Search ...

Search

Join Our WhatsApp Channel

Join Our Training Program

Join Our Telegram Channel

Join Our Discord Channel

Cyber Security Mindmap



Follow Us On Twitter



Follow Us On Linkedin



You may like

A Detailed Guide on Nmap Firewall Scan

May 20, 2026

Penetration Testing on MYSQL (Port 3306)

August 6, 2024
